

Developer Handoff

Prepaid USDC credits, lookup pricing, ledger, and Hostinger deployment

Flow

- User creates and verifies an account.
- User selects a credit package.
- GrayOcean generates an exact USDC payment request.
- Payment is confirmed on one supported network.
- The platform credits the user's internal balance.
- Each completed lookup deducts the applicable amount.
- Credits cannot be withdrawn, transferred, or converted back to USDC.

This distinction matters. Calling it a “wallet” can imply that GrayOcean is providing custody or money-transmission functionality. Call it:

- Credit balance
- Account balance
- Prepaid usage balance

Not “crypto wallet.”

Pricing problem

At \$0.10–\$0.15 per search, a \$100 deposit provides:

- 1,000 searches at \$0.10
- 800 searches at \$0.125
- 667 searches at \$0.15

That may work if your underlying data cost is extremely low. But the site currently promises substantially different products—SSN validation, address intelligence, skiptrace, and batch analysis. They should probably not all carry the same price.

A cleaner initial structure:

Search type	Suggested retail
SSN validation	\$0.10
Address verification	\$0.10
Address history/enrichment	\$0.15
Basic skiptrace	\$0.25
Enhanced skiptrace	\$0.50–\$1.00
Combined identity report	\$0.50
Batch lookup	Same unit price, volume tiers later

Your actual pricing must be based on the vendor cost per successful record, failed-query charges, infrastructure cost, fraud losses, and desired gross margin.

I would target a minimum 70% gross margin before locking prices.

Do not require a permanent \$100 balance

There are two different concepts:

Better approach

Minimum initial deposit: \$100

After funding, users can continue searching until their credit balance reaches the cost of the next lookup. When the balance gets low, prompt them to reload.

More restrictive approach

Minimum available balance: \$100 at all times

This means the user deposits \$100 but cannot spend below \$100. That creates trapped capital and makes the product feel hostile. It also means a user would need to deposit \$200 to actually consume \$100 of searches.

Use a \$100 minimum purchase, not a permanent \$100 minimum balance.

You could add:

- Minimum initial credit purchase: \$100
- Minimum reload: \$50 or \$100

Recommended packages

Do not make users type arbitrary payment amounts at launch.

Starter

\$100 in credits

For independent investigators and evaluation.

Team

\$500 in credits

Could include a 3%–5% bonus after margins are verified.

Operations

\$1,000 in credits

Could include priority batch processing or a small credit bonus.

Enterprise

Custom invoicing, higher limits, API, team controls, and negotiated pricing.

Avoid aggressive discounts until you understand vendor costs and abuse rates.

Payment architecture

Use USDC on one network only at launch. Display the network everywhere:

USDC on Base only

Do not send assets using another network.

Each order should contain:

- Unique order ID
- User ID
- USD credit amount
- Exact USDC amount
- Receiving address or payment destination
- Supported network
- Created time
- Expiration time
- Transaction hash
- Block confirmation status
- Credited time
- Credited amount

The transaction hash must have a unique database constraint so it can never fund two accounts.

Possible statuses:

- Awaiting payment
- Payment detected
- Confirming
- Credited
- Underpaid
- Overpaid
- Expired
- Wrong asset
- Wrong network
- Manual review
- Rejected

USDC payments still require sanctions controls. OFAC states that U.S. businesses have the same sanctions obligations for digital-currency transactions as for traditional payments and recommends a risk-based screening program.

Ledger structure

Never update a balance without creating a ledger transaction.

Every account movement should produce an immutable record:

- credit_purchase
- lookup_charge
- lookup_reversal
- promotional_credit
- manual_adjustment
- refund_adjustment
- expired_credit, only if your terms permit expiration

Each lookup record should include:

- User
- Organization
- Search type
- Input reference, securely masked
- Vendor used
- Vendor request ID
- Search status
- Retail charge
- Vendor cost
- Margin
- Timestamp
- Analyst or API key
- Reason or permissible-purpose code
- Result audit reference

Use an append-only ledger. The visible balance should be calculated from ledger entries or updated transactionally with the ledger.

Charge only for valid outcomes

You need clear rules for failed searches.

Recommended:

- System error: no charge
- Vendor timeout: no charge or automatic reversal
- No match: charge only if the upstream provider charges and the user was warned
- Invalid input detected before vendor request: no charge
- Successful response with limited data: charge according to the disclosed lookup type
- Duplicate accidental submission: consider a short deduplication window

Display the price before the user clicks Run search.

Example:

SSN validation

Cost: 0.10 credits

Available balance: \$97.40

Then show the new balance immediately afterward.

Hostinger deployment

Hostinger can support the application, but I would use a Hostinger VPS, not basic shared hosting. Hostinger's VPS supports Docker, PostgreSQL, Node.js and custom server configurations; the VPS also gives the developer root access.